

FULL

STACK

Full-Stack

Arsitektur Full-Stack

Menggabungkan HTML/JS frontend dengan Express+Prisma backend

Konsep Full-Stack Development

Tiga layer yang bekerja sama: Frontend ↔ Backend ↔ Database

① FRONTEND

HTML Struktur halaman — elemen, form, tombol

CSS Tampilan & styling — warna, layout, animasi

JavaScript Logika UI — event, manipulasi DOM

Fetch API Kirim HTTP request ke backend

DOM API Render data dari server ke halaman

Berjalan di Browser pengguna (Chrome, Firefox, dll)

Port http://localhost:5500

② BACKEND

Node.js Runtime — JavaScript di luar browser

Express.js Framework — routing, middleware, API

Dotenv Baca config dari file .env

REST API Endpoint yang dipanggil frontend

Middleware CORS, parsing JSON, autentikasi

Berjalan di Server (local atau cloud)

Port http://localhost:3000



③ DATABASE

SQLite Database ringan — 1 file, tanpa install server

Prisma ORM Jembatan JavaScript ↔ Database

schema.prisma Definisi model/tabel

migrate dev Buat/update tabel otomatis

PrismaClient Objek untuk query: findMany, create...

Disimpan di File prisma/dev.db di dalam server



Format komunikasi Frontend ↔ Backend: **JSON** Contoh: { "id": 1, "text": "Belajar Express", "done": false }

HTTP & REST API

Cara frontend berbicara dengan backend — method, endpoint, request & response

APA ITU REST API?

REST Representational State Transfer — gaya arsitektur API

HTTP HyperText Transfer Protocol — bahasa komunikasi web

REST API Kontrak komunikasi antar sistem menggunakan HTTP + JSON

HTTP METHODS — 4 Operasi Utama

METHOD	ENDPOINT	STATUS	KETERANGAN
GET	/api/todos	200 OK	Ambil semua data — tidak
POST	/api/todos	201	Buat data baru — body: {
PATCH	/api/todos/:id	200 OK	Update sebagian — body: {
DELETE	/api/todos/:id	204 No Content	Hapus data — tidak ada request body

ANATOMI HTTP REQUEST

Method + URL POST http://localhost:3000/api/todos

Header wajib Content-Type: application/json

Header opsional Authorization: Bearer <token>

Body (POST/PATCH) { "text": "Belajar REST API" }

Tanpa body GET dan DELETE tidak perlu body

ANATOMI HTTP RESPONSE

Status code 200, 201, 400, 404, 500 (lihat tabel kiri)

Body { "id": 1, "text": "...", "done": false }

2xx Sukses — 200 OK, 201 Created, 204 No Content

4xx Error dari client — 400 Bad Request, 404 Not Found

5xx Error dari server — 500 Internal Server Error

APA ITU EXPRESS.JS?

Node.js Runtime JavaScript di server — menjalankan JS di luar browser

Express.js Library web framework di atas Node.js — menyederhanakan server

Tanpa Express Perlu tulis HTTP parsing, routing, error handling manual

Dengan Express Routing 1 baris, middleware mudah, ekosistem besar

ALUR MIDDLEWARE PIPELINE

→ REQUEST dari browser

→ `cors()` — izinkan request lintas domain

→ `express.json()` — parse body JSON

→ `authMiddleware()` — cek token (opsional)

→ Route Handler — logika GET / POST / dll

→ RESPONSE dikirim ke browser

```
backend/index.js (minimal Express server)

const express = require('express');
const cors = require('cors');

const app = express();

// Middleware dijalankan berurutan setiap request
app.use(cors({ origin: '*' }));
app.use(express.json()); // baca req.body

// Route: tangani GET ke URL "/"
app.get('/', (req, res) => {
  res.json({ message: 'Halo!' });
});

// Route: daftarkan semua endpoint /api/todos
app.use('/api/todos', require('./routes/todos'));
```

KONSEP KUNCI

`app.use(fn)` Daftarkan middleware — dijalankan setiap request

`app.get/post/patch/delete(url, fn)` Route spesifik

`req` Object request: `req.body`, `req.params`, `req.headers`

`res` Object response: `res.json()`, `res.status()`

`next()` Teruskan ke middleware berikutnya

Prisma ORM & SQLite | Database Tanpa Ribet

Object-Relational Mapping — tulis JavaScript, Prisma yang urus SQL-nya

APA ITU ORM?

ORM Object-Relational Mapping — jembatan kode JS dan tabel database

Tujuan Tulis query dalam bahasa JavaScript, bukan SQL mentah

Prisma ORM modern untuk Node.js — type-safe, auto-generate client

TANPA ORM — SQL manual

```
// Query manual — rentan typo & SQL injection
db.query("SELECT * FROM todos ORDER BY created_at DESC")
db.query("INSERT INTO todos (text, done) VALUES (?, ?)", [text, false])
db.query("UPDATE todos SET done=? WHERE id=?", [done, id])
// Perlu sanitize manual, tidak ada type checking
```

DENGAN PRISMA — JavaScript saja

```
// Otomatis aman, auto-complete, type-safe
prisma.todo.findMany({ orderBy: { createdAt: 'desc' } })
prisma.todo.create({ data: { text: text.trim() } })
prisma.todo.update({ where: { id }, data: { done } })
prisma.todo.delete({ where: { id } })
// Prisma handle sanitize & parameterized query otomatis!
```

PRISMA WORKFLOW

- ① Edit prisma/schema.prisma — definisi model & tipe field
- ② `npx prisma migrate dev --name init` — buat tabel di DB
- ③ Prisma Client otomatis di-generate (jangan edit manual)
- ④ Import & pakai: `const prisma = require("./db")`

findMany() → SELECT semua baris — returns array

findUnique() → SELECT satu baris by id

create() → INSERT baris baru

update() → UPDATE baris by id

delete() → DELETE baris by id

MENGAPA SQLite UNTUK BELAJAR?

Zero install Tidak perlu install MySQL/PostgreSQL/Docker

File tunggal `dev.db` — mudah di-reset & dibagikan ke teman

Cukup powerful Untuk ratusan ribu baris data — lebih dari cukup

Production? Ganti ke PostgreSQL saat deploy — cukup ubah `schema.prisma`

CORS | Cross-Origin Resource Sharing

Mengapa browser memblokir request lintas domain — dan cara mengizinkannya

MASALAH — Same-Origin Policy (SOP)

Browser punya aturan keamanan bawaan:

Same-Origin Policy (SOP)

"Origin" = protokol + domain + port

Frontend berjalan di:

`http://localhost:5500`

Backend berjalan di:

`http://localhost:3000`

Beda PORT = Beda Origin → Browser BLOKIR!

Error yang muncul di DevTools (F12):

```
Access to fetch at
'http://localhost:3000/api/todos'
from origin
'http://localhost:5500' has been
blocked by CORS policy:
No 'Access-Control-Allow-Origin'
header is present on the requested resource.
```

Ini bukan bug — ini fitur keamanan browser
yang melindungi user dari serangan web!

SOLUSI — Enable CORS di Backend

Cara kerja Backend kirim header khusus ke browser:

Header Access-Control-Allow-Origin: `http://localhost:5500`

Lalu Browser baca header → izinkan request!

backend/index.js — aktifkan CORS (3 langkah)

```
// LANGKAH 1 — install package cors
// Di terminal: npm install cors

// LANGKAH 2 — import
const cors = require('cors');

// LANGKAH 3 — pasang sebagai middleware
app.use(cors({
  origin: process.env.FRONTEND_URL || '*',
  credentials: true
}));
```

CATATAN PENTING

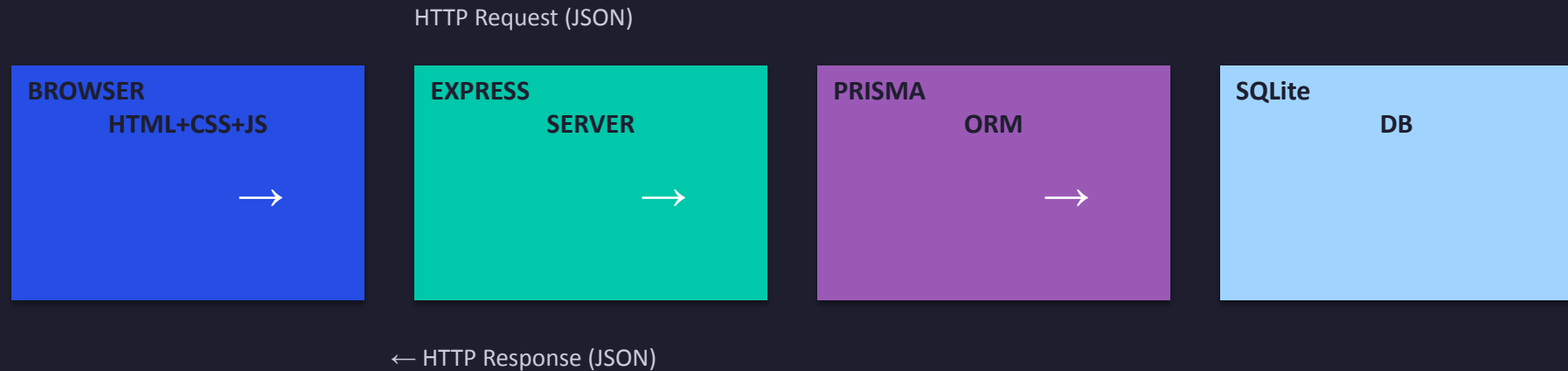
origin: '*' izinkan SEMUA domain — OK untuk development

Production ganti '*' dengan domain frontend yang spesifik

credentials wajib true jika pakai cookie atau Authorization header

Gambaran Arsitektur & CORS

Alur data dari browser ke database dan kembali



```
// Backend: izinkan frontend akses API
npm install cors

// index.js
app.use(require('cors') ({
  origin: process.env.FRONTEND_URL ||
  '*',
  credentials: true
})));

// .env
FRONTEND_URL=http://localhost:5500
JWT_SECRET=rahasia_panjang
PORT=3000
```


Frontend HTML+JS Fetch ke Express

Vanilla JS murni — fetch API + token di localStorage

```
// frontend/app.js
const API = 'http://localhost:3000/api';
let token = localStorage.getItem('token');

// Helper fetch dengan auth header
async function apiFetch(path, opts={}) {
  const res = await fetch(API + path, {
    ...opts,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? {Authorization: 'Bearer '+token} : {})
    }
  });
  if (res.status === 401) {
    localStorage.removeItem('token'); token=null;
    showLogin(); return;
  }
  return res.json();
}

// Login
async function login(email, password) {
  const data = await apiFetch('/auth/login', {
    method: 'POST',
    body: JSON.stringify({email, password})
  });
  if (data?.token) {
    token = data.token;
    localStorage.setItem('token', token);
    showDashboard();
  }
}
```

```
// Ambil dan tampilkan produk
async function loadProducts() {
  const grid = document.querySelector('#grid');
  grid.innerHTML = '<p>Loading...</p>';
  try {
    const data = await apiFetch('/products');
    grid.innerHTML = data.map(p => `
      <div class='card'>
        <h3>${p.nama}</h3>
        <p>Rp${p.harga.toLocaleString()}</p>
        <p>Stok: ${p.stok}</p>
        <button onclick='deleteProduct(${p.id})'>Hapus</button>
      </div>`).join('');
  } catch (e) {
    grid.innerHTML = '<p>Gagal memuat data</p>';
  }
}

async function deleteProduct(id) {
  await apiFetch('/products/'+id, {method: 'DELETE'});
  loadProducts(); // refresh
}

// Panggil saat halaman siap
document.addEventListener('DOMContentLoaded', () => {
  token ? loadProducts() : showLogin();
});
```

Deployment

Dari localhost ke production — Real server di internet

| PM2

Process manager Node.js

```
# PM2 - jalankan Node.js di server
npm install -g pm2
pm2 start index.js --name my-api
pm2 status          # lihat proses
pm2 logs my-api     # lihat log
pm2 restart my-api
pm2 startup && pm2 save # auto-start boot

# package.json - tambahkan engines
{
  "scripts": {
    "start": "node index.js"
  },
  "engines": { "node": ">=18" }
}
```

Checklist Sebelum Deploy

- package.json punya script 'start'
- engines.node sudah diset
- Semua config dari .env (tidak hardcode)
- CORS origin sudah diupdate ke domain frontend
- .gitignore: .env dan node_modules tidak ter-commit

Praktik

Bangun dan deploy aplikasi full-stack

PRAKTIK | Struktur Folder Project

Pahami struktur sebelum mulai menulis kode

FOLDER STRUCTURE

todo-fullstack/

backend/

routes/

todos.js <- CRUD endpoint

middleware/

prisma/

schema.prisma

index.js <- entry point server

db.js <- Prisma client

.env <- JANGAN commit!

.gitignore

package.json

frontend/

index.html

style.css

app.js <- fetch ke backend

PRASYARAT

- ✓ Node.js versi 18+ → node -v
- ✓ npm versi 8+ → npm -v
- ✓ VS Code + ekstensi Live Server
- ✓ Terminal (CMD / PowerShell / Git Bash)
- PM2 (opsional — process manager)

PENJELASAN FILE

index.js — Entry point, setup Express+CORS

db.js — Prisma Client singleton

routes/todos.js — Semua CRUD /api/todos

schema.prisma — Definisi tabel database

.env — Secret config, JANGAN di-commit

.gitignore — File yang diabaikan Git

frontend/app.js — Fetch API, render todo list

Step 1 | Buat Folder Project & Install Dependencies

Jalankan semua perintah ini secara berurutan di Terminal

```
Terminal / CMD / PowerShell

# 1. Buat folder utama project
mkdir todo-fullstack
cd todo-fullstack
mkdir backend frontend

# 2. Masuk ke folder backend
cd backend

# 3. Inisialisasi package.json (--y = semua jawaban default)
npm init -y

# 4. Install library yang dibutuhkan (production)
npm install express cors dotenv @prisma/client
#   express      -> web server framework
#   cors         -> izinkan akses lintas domain (browser)
#   dotenv       -> baca variabel dari file .env
#   @prisma/client -> ORM untuk query database

# 5. Install Prisma CLI sebagai devDependency
npm install -D prisma

# 6. Init Prisma -> buat folder prisma/ + file .env secara otomatis
npx prisma init --datasource-provider sqlite

Notes:
- Jika error saat menjalankan npx prisma init --datasource-provider sqlite, perlu upgrade terlebih dahulu karena perbedaan version nodejs ( node v26.3.0 Prisma to v6 ). Contoh error : Error: (0 , CSe.isError) is not a function
- Upgrade : npm install prisma@latest @prisma/client@latest --save lalu jalankan lagi npx prisma init --datasource-provider sqlite

# Hasil akhir yang terbentuk di folder backend/
#   node_modules/      <- hasil npm install (JANGAN di-commit)
#   prisma/schema.prisma <- template schema, perlu kita edit
#   .env               <- berisi DATABASE_URL, perlu kita isi lengkap
#   package.json       <- daftar dependencies
```

Step 2 | Konfigurasi .env & .gitignore

PRAKTIK

Lakukan sebelum menulis code - ini soal keamanan

backend/.env (isi sesuai ini)

```
# Konfigurasi environment – JANGAN pernah commit file ini!

# URL database SQLite (file lokal di folder prisma/)
DATABASE_URL="file:./prisma/dev.db"

# Secret untuk tanda tangan JWT – wajib panjang & acak
JWT_SECRET="ganti_dengan_32_karakter_acak_panjang"

# Port server Express
PORT=3000

# Domain frontend yang boleh akses backend (CORS)
FRONTEND_URL="http://localhost:5500"

# ——— CARA BUAT JWT_SECRET YANG AMAN ———
# Buka terminal baru, jalankan:
node -e "console.log(
  require('crypto')
    .randomBytes(32).toString('hex'))"
# Salin output (64 hex chars) ke JWT_SECRET di atas
```

backend/.gitignore

```
# Wajib ada sebelum git init!

# Jangan commit node_modules (besar, bisa di-install ulang)
node_modules/

# WAJIB: jangan bocorkan secret
.env

# File database SQLite (data lokal)
prisma/dev.db
prisma/dev.db-journal
```

⚠ PENTING: Urutan yang benar

1. Buat .gitignore (langkah ini dulu!)
2. Buat .env dari template di atas
3. Baru jalankan: git init

Jika .env ter-commit ke GitHub:

- Semua password & token bocor ke publik
- Wajib ganti semua secret seketika
- GitHub sendiri akan kirim peringatan

Step 3 | Prisma Schema & Generate Database

PRAKTIK

Edit prisma/schema.prisma lalu jalankan migration

```
backend/prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Todo {
  id        Int      @id @default(autoincrement())
  text      String
  done      Boolean   @default(false)
  createdAt DateTime @default(now())
}
```

```
Terminal (di folder backend/) - Buat database

# Jalankan migration: buat tabel di database SQLite
npx prisma migrate dev --name init
Notes :
- Jika add error, change prisma version dengan command :
  - npm install @prisma/client@5 && npm install -D prisma@5
  - npx prisma migrate dev --name init

# Output sukses:
# ✓ Generated Prisma Client
# ✓ Applied migration 'init'

# Buka GUI untuk lihat isi database:
npx prisma studio
# → buka http://localhost:5555 di browser
```

APA YANG TERJADI?

- ① Prisma baca schema.prisma → buat SQL CREATE TABLE
- ② File dev.db (database) otomatis terbentuk di prisma/
- ③ Prisma generate Prisma Client (TypeScript types)
- ④ Setiap kali schema berubah → jalankan migrate dev lagi
- ⑤ Migration disimpan di prisma/migrations/ (jangan hapus!)

Step 4 | Backend — index.js & db.js

PRAKTIK

Buat dua file ini di folder backend/

backend/index.js

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');

const app = express();

// Izinkan akses dari domain frontend (CORS)
app.use(cors({
  origin: process.env.FRONTEND_URL || '*',
  credentials: true
}));

// Agar bisa baca req.body berformat JSON
app.use(express.json());

// Health check — test apakah server sudah jalan
app.get('/', (req, res) => res.json({ status: 'ok', app: 'todo-api' }));

// Daftarkan routes — semua endpoint /api/todos ada di sini
app.use('/api/todos', require('./routes/todos'));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`✓ Server jalan di http://localhost:${PORT}`);
});
```

backend/db.js

```
// Singleton: satu koneksi untuk seluruh app
// Import di route: const prisma = require("../db")

const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient();

module.exports = prisma;
```

backend/package.json (update bagian scripts)

```
"scripts": {
  "start": "node index.js",
  "dev" : "node --watch index.js"
},
"engines": {
  "node": ">=18"
}
```

Step 5 | routes/todos.js — GET & POST

PRAKTIK

Buat folder routes/ lalu file todos.js di dalamnya

```
backend/routes/todos.js (bagian 1)

const router = require('express').Router();
const prisma = require('../db');

// GET /api/todos - ambil semua todo, terbaru tampil pertama
router.get('/', async (req, res) => {
  try {
    const todos = await prisma.todo.findMany({
      orderBy: { createdAt: 'desc' }
    });
    res.json(todos);
  } catch (e) {
    res.status(500).json({ message: e.message });
  }
});

// POST /api/todos - buat todo baru dari req.body.text
router.post('/', async (req, res) => {
  const { text } = req.body;
  if (!text?.trim()) {
    return res.status(400).json({ message: 'Text kosong' });
  }
  try {
    const todo = await prisma.todo.create({ data: { text: text.trim() } });
  } catch (e) {
    res.status(500).json({ message: e.message });
  }
});
```

CATATAN PENTING

try/catch → selalu wrap kode Prisma, error tidak crash server

res.json() → kirim response berformat JSON ke frontend

200 OK → request berhasil, data dikembalikan

201 Created → data baru berhasil dibuat

400 Bad Request → input dari client tidak valid

500 Server Error → terjadi error di sisi server

text?.trim() → optional chaining: aman meski text undefined/null

await → tunggu query Prisma selesai sebelum lanjut

req.body → data yang dikirim frontend via body JSON

findMany() → ambil banyak data (array)

create() → insert satu baris baru ke database

🚩 WAJIB di akhir file:

module.exports = router;

Step 6 | routes/todos.js — PATCH & DELETE + module.exports

PRAKTIK

Lanjutan file routes/todos.js — tambahkan di bawah kode POST

```
backend/routes/todos.js (bagian 2 – lanjutan)

// PATCH /api/todos/:id – toggle done true/false
router.patch('/:id', async (req, res) => {
  const id = parseInt(req.params.id, 10);
  try {
    const todo = await prisma.todo.update({
      where: { id },
      data: { done: req.body.done }
    });
    res.json(todo);
  } catch (_) {
    res.status(404).json({ message: 'Tidak ditemukan' });
  }
});

// DELETE /api/todos/:id – hapus satu todo
router.delete('/:id', async (req, res) => {
  const id = parseInt(req.params.id, 10);
  try {
    await prisma.todo.delete({ where: { id } });
    res.status(204).send(); // 204 = No Content (berhasil, tidak ada data)
  } catch (_) {
    res.status(404).json({ message: 'Tidak ditemukan' });
  }
});

// WAJIB di paling bawah file!
module.exports = router;
```

RINGKASAN SEMUA ENDPOINT

GET	/api/todos	200	Array semua todo
POST	/api/todos	201	Todo baru, kirim { text }
PATCH	/api/todos/:id	200	Update done, kirim { done }
DELETE	/api/todos/:id	204	Hapus, tidak ada body

TIPS PENTING

req.params.id → string! Perlu parseInt() jadi number

update() → ubah data yang sudah ada di database

delete() → hapus data dari database permanen

where: { id } → shorthand dari where: { id: id }

204 No Content → berhasil, tapi tidak ada data dikembalikan

catch (_) → _ artinya error diabaikan (kita tahu itu 404)

Step 7 | Frontend — index.html & app.js

PRAKTIK

Buat 3 file di folder frontend/ (pisah dari backend/)

frontend/index.html

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Todo App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="app">
    <h1>Todo App</h1>
    <div class="add-form">
      <input type="text" id="inp"
        placeholder="Tulis todo baru..."
        onkeydown="if(event.key==='Enter') addTodo()">
      <button onclick="addTodo()">Tambah</button>
    </div>
    <ul id="list"></ul>
  </div>
  <script src="app.js"></script>
</body>
</html>
```

Step 7 | Frontend — index.html & app.js

PRAKTIK

Buat 3 file di folder frontend/ (pisah dari backend/)

frontend/app.js (apiFetch + loadTodos)

```
const API = 'http://localhost:3000/api';

async function apiFetch(path, opts={}) {
  const res = await fetch(API+path, { headers:
    {'Content-Type':'application/json'}, ...opts });
  if (res.status===204) return null;
  const data = await res.json();
  if (!res.ok) throw new Error(data.message);
  return data;
}
```

frontend/app.js (loadTodos, addTodo, toggleTodo, deleteTodo)

```
async function loadTodos() {
  const list = document.querySelector('#list');
  list.innerHTML = '<li class="loading">Memuat...</li>';
  try {
    const todos = await apiFetch('/todos');
    if (!todos.length)
      return list.innerHTML = '<li>Belum ada todo!</li>';
    list.innerHTML = todos.map(t=>`
      <li class="${t.done?'done':''}">
        <label><input type="checkbox"
          ${t.done?'checked':''} onchange=
            "toggleTodo(${t.id},this.checked)">
          <span>${t.text}</span></label>
          <button onclick="deleteTodo(${t.id})">Hapus</button>
        </li>`).join('');
  } catch(e) { list.innerHTML = `<li>Gagal: ${e.message}</li>`; }
}

async function addTodo() {
  const inp=document.querySelector('#inp');
  const text=inp.value.trim();
  if (!text) return;
  await apiFetch('/todos', { method:'POST',
    body:JSON.stringify({text}) });
  inp.value=''; loadTodos();
}

async function toggleTodo(id,done) {
  await apiFetch(`/todos/${id}`, { method:'PATCH',
    body:JSON.stringify({done}) }); loadTodos();
}

async function deleteTodo(id) {
  if (!confirm('Hapus?')) return;
  await apiFetch(`/todos/${id}`, { method:'DELETE' }); loadTodos();
}

document.addEventListener('DOMContentLoaded', loadTodos);
```

Step 8 | Jalankan Aplikasi & Test

PRAKTIK

Jalankan backend di Terminal 1, frontend di Terminal 2, test di browser

Terminal 1 – Start Backend

```
# Pastikan berada di folder backend/  
cd backend  
  
# Cara A: langsung (perlu tetap terbuka)  
node index.js  
# ✓ Server jalan di http://localhost:3000  
  
# Cara B: dev mode (auto restart saat file berubah)  
npm run dev  
  
# Cara C: PM2 – background, auto restart jika crash  
pm2 start index.js --name todo-api  
pm2 logs todo-api # lihat log real-time  
pm2 restart todo-api # restart setelah edit kode  
  
# Windows – install PM2 dulu:  
npm install -g pm2
```

Terminal 2 – Start Frontend

```
# Cara A: VS Code Live Server (paling mudah)  
# Klik kanan index.html → "Open with Live Server"  
# Browser otomatis buka: http://localhost:5500  
  
# Cara B: http-server di terminal  
npm install -g http-server  
cd frontend  
http-server -p 5500
```

Test endpoint – Browser atau curl

```
# Test server hidup  
curl http://localhost:3000/  
# → {"status":"ok","app":"todo-api"}  
  
# Test GET semua todo  
curl http://localhost:3000/api/todos  
# → [] (kosong di awal, normal)  
  
# Test POST (buat todo baru)  
curl -X POST http://localhost:3000/api/todos \  
-H 'Content-Type: application/json' \  
-d '{"text":"Belajar Express"}'  
# → {"id":1,"text":"Belajar Express","done":false,...}
```

JIKA ADA MASALAH

Port 3000 dipakai → ganti PORT=3001 di .env

CORS error browser → cek FRONTEND_URL di .env backend

Cannot find 'prisma' → cd backend && npm install

DATABASE_URL not found → buat file .env dari .env.example

Todo tidak muncul → buka DevTools F12, cek tab Console

Roadmap Selanjutnya — Terus Berkembang!

Selesai pertemuan — ini baru permulaan!

React / Vue

Component, State, Hooks → SPA modern → Vite + TypeScript

Next.js

SSR + SSG + App Router → SEO friendly → Deploy Vercel

Database

PostgreSQL + Prisma relasi kompleks → Redis caching

Docker

Containerize app → docker-compose → Kubernetes dasar

Testing

Jest unit test → Supertest API test → Cypress E2E

CI/CD Lanjut

GitHub Actions lengkap → staging & production environment

Selamat! Kamu sudah membangun pondasi web developer yang kuat.